

Competitive Programming

Section 53 Editorial: Advanced Combinatorics

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
1.1	Detailed solution guide	3
2	A. Catalan Queries	4
2.1	Restatement	4
2.2	Observations and algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	4
3	B. Derangement Queries	6
3.1	Restatement	6
3.2	Observations and algorithm	6
3.3	Correctness proof	6
3.4	Complexity	6
3.5	Full C++ solution	6
3.6	Full Python solution	6
4	C. Necklace Orbits	8
4.1	Restatement	8
4.2	Observations and algorithm	8
4.3	Correctness proof	8
4.4	Complexity	8
4.5	Full C++ solution	8
4.6	Full Python solution	8
5	What to carry forward	10

1 Editorial overview

1.1 Detailed solution guide

A. Catalan Numbers. The number of balanced parenthesis strings with n pairs is $C_n = \text{binom}(2n, n) / (n+1)$. The binomial chooses the positions of the opening brackets, and the division removes the invalid paths counted by the reflection argument. With a prime modulus, compute factorials and inverse factorials, then multiply by the modular inverse of $n+1$.

B. Derangements. Let $D[n]$ be permutations of n elements with no fixed points. Look at where element 1 goes, say to j . Either j maps back to 1, leaving $D[n-2]$ choices, or j does not map back, which can be converted to a derangement-like arrangement on $n-1$ elements. This gives $D[n] = (n-1)(D[n-1] + D[n-2])$.

C. Burnside Rotations. Burnside's lemma says the number of necklaces is the average number of colorings fixed by each rotation. A rotation by r positions has $\text{gcd}(n, r)$ cycles, and each cycle must have one color, so it fixes $k^{\text{gcd}(n, r)}$ colorings. Sum this over all rotations and divide by n .

Common pitfalls. Modular division means multiplying by an inverse. Burnside counts orbits, not representatives you construct manually. Derangement bases are $D[0]=1$, $D[1]=0$.

This section is about selecting the correct counting theorem before coding.

- A evaluates Catalan numbers with factorials and modular inverses: $C(2n, n) / (n+1)$.
- B precomputes derangements using $D[n] = (n-1) * (D[n-1] + D[n-2])$.
- C applies Burnside's lemma to rotations of a necklace: average $k^{\text{gcd}(n, r)}$ over all rotations r .

The correctness arguments identify the counted structure and then cite the corresponding recurrence or orbit-counting lemma with its assumptions.

2 A. Catalan Queries

2.1 Restatement

Print Catalan numbers modulo $1e9+7$.

2.2 Observations and algorithm

Use the invariant named in the editorial overview. The code processes the input in the natural order and updates exactly the state described by that invariant, so each branch corresponds to one mathematical case rather than an implementation trick.

2.3 Correctness proof

Correctness follows from the invariant proved in the detailed guide for this problem. The code initializes the base state, applies exactly the transition described there, and prints the value represented by the final state. $\square$

2.4 Complexity

The complexity follows from the concrete loops and data structures in the code below; the detailed guide also indicates which optimized structure is needed for larger external versions.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; const long long MOD=1000000007; long long pw(long long a,long long e){l
```

2.6 Full Python solution

```
import sys
MOD=1_000_000_007
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    qs=data[1:1+data[0]]; N=max(qs+[0])*2; fact=[1]*(N+1)
```

```

for i in range(1,N+1): fact[i]=fact[i-1]*i%MOD
inv=[1]*(N+1); inv[N]=pow(fact[N],MOD-2,MOD)
for i in range(N,0,-1): inv[i-1]=inv[i]*i%MOD
def C(n,k): return fact[n]*inv[k]%MOD*inv[n-k]%MOD if 0<=k<=n else 0
print("\n".join(str(C(2*n,n)*pow(n+1,MOD-2,MOD)%MOD) for n in qs))
if __name__=="__main__": main()

```

3 B. Derangement Queries

3.1 Restatement

Print the number of derangements of n modulo $1e9+7$.

3.2 Observations and algorithm

Use the invariant named in the editorial overview. The code processes the input in the natural order and updates exactly the state described by that invariant, so each branch corresponds to one mathematical case rather than an implementation trick.

3.3 Correctness proof

Correctness follows from the invariant proved in the detailed guide for this problem. The code initializes the base state, applies exactly the transition described there, and prints the value represented by the final state. $\square$

3.4 Complexity

The complexity follows from the concrete loops and data structures in the code below; the detailed guide also indicates which optimized structure is needed for larger external versions.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; const long long MOD=1000000007; int main(){ios::sync_with_stdio(false);
```

3.6 Full Python solution

```
import sys
MOD=1_000_000_007
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    qs=data[1:1+data[0]]; N=max(qs+[1]); d=[0]*(N+1); d[0]=1
```

```
if N>=1: d[1]=0
for i in range(2,N+1): d[i]=(i-1)*(d[i-1]+d[i-2]))%MOD
print("\n".join(str(d[n]) for n in qs))
if __name__=="__main__": main()
```

4 C. Necklace Orbits

4.1 Restatement

Count color necklaces of length n with k colors up to rotation.

4.2 Observations and algorithm

Use the invariant named in the editorial overview. The code processes the input in the natural order and updates exactly the state described by that invariant, so each branch corresponds to one mathematical case rather than an implementation trick.

4.3 Correctness proof

Correctness follows from the invariant proved in the detailed guide for this problem. The code initializes the base state, applies exactly the transition described there, and prints the value represented by the final state. $\square$

4.4 Complexity

The complexity follows from the concrete loops and data structures in the code below; the detailed guide also indicates which optimized structure is needed for larger external versions.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; const long long MOD=1000000007; long long pw(long long a,long long e){l
```

4.6 Full Python solution

```
import sys,math
MOD=1_000_000_007
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    t=data[0]; idx=1; out=[]
```

```
for _ in range(t):
    n,k=data[idx],data[idx+1]; idx+=2; s=0
    for r in range(n): s=(s+pow(k,math.gcd(n,r),MOD))%MOD
    out.append(str(s*pow(n,MOD-2,MOD)%MOD))
print("\n".join(out))
if __name__=="__main__": main()
```

5 What to carry forward

Red-level improvement comes from making the invariant explicit, stress-testing it, and then upsolving the stronger external versions.